TEN WEEKS · PROJECT-BASED

Agentic Engineer

Study Guide 2026

Updated September 4, 2026 · 198 checked links

https://agentic-engineer-study-guide.vercel.app

A ten-week, project-based guide to building software with coding agents: how agents work inside, context engineering and MCP, reusable skills, repository readiness, AI code review, security, background agents, team adoption, and the software factory. Each week pairs a capped set of core material with a build and a checkable “done when” list, so you finish with a working system rather than a reading log.

Credit. The ten-week structure and the weekly topics follow the Fall 2026 syllabus of Stanford's CS146S, *The Modern Software Developer* (themodernsoftware.dev). This guide is independent and not affiliated with the course; every reading, video, build, and checklist is its own selection. All links were checked on September 4, 2026, and every video carries its running time so you can budget honestly.

The ten weeks at a glance

1. **The Internals of Coding Agents**
   A 300–500 line terminal agent with four tools and full logging
2. **Advanced Context Engineering**
   A one-page spec, one feature shipped through RePPIT, and an MCP server with 2–4 tools
3. **Agent Skills and CLI**
   A packaged skill with a helper script, plus a browser-driven web skill
4. **Customizing Your Agent and Repository**
   Repository instructions, two hooks, and a planner / implementer / reviewer split
5. **Agent-Ready Codebases**
   A scored readiness audit and the fixes that make a fresh agent productive

6 **Agentic Code Review**
A severity-ordered review rubric wired to pull requests, measured on five PRs

7 **Security**
A threat model, SAST / SCA / secret scans in CI, and a prompt-injection test

8 **Background Agents**
An issue-to-PR flow with isolation, budgets, checkpoints, and retries

9 **Building an AI-Native Team**
A model gateway, an MCP portal, and a one-page adoption policy

10 **The Software Factory + The Future**
A traced end-to-end factory with an eval set and a controlled improvement loop

How to use this guide

Plan on **10–12 hours per week**. Split it like this:

3–4 hours: the **Core material** for the week. Each week's core list is capped near four hours; the running times are printed next to each item.

5–6 hours: the weekly **Build**.

1–2 hours: document results, failures, costs, and lessons, then tick the week's **Done when** list.

Everything under **Deeper material**, **Additional video track**, and **Tools and references** is optional. Pick by interest; do not try to clear it.

Use one medium-sized repository throughout the ten weeks. A small SaaS app, developer tool, or API with a UI, tests, and CI works better than ten disconnected toy projects. Each week should improve the same system.

The strongest free backbone is Hugging Face's Context Course (units: Agent Skills, MCP, Plugins, Sub-agents, Hooks, and a bonus Nano Harness). It overlaps unusually well with the first half of this guide. Use the materials below to deepen and extend it.

The guide is language-agnostic; most examples use Python or JavaScript. You need solid programming experience (two or three university-level courses or the equivalent). A machine-

learning background helps but is not required; if transformers are new to you, the Week 1 optional foundation covers the gap.

Tooling and budget

Some tools need a subscription or an API key. Decide this before Week 1:

Link

Claude Code

Pick one primary coding agent and stay with it for at least five weeks so the customization work in Weeks 3–4 compounds. Reasonable choices

(subscription or API key), OpenAI Codex CLI (open source, subscription or API key), Cursor, Gemini CLI (open source, has a free tier), or opencode (open source, bring your own key).

Set a hard monthly spend cap before you start and log spend per week alongside your build notes. Cost is a first-class metric in this guide, not an afterthought.

Link

prompt caching

**Learn

** and use a cheaper model for subagents and review passes. Most runaway bills come from long contexts re-sent on every turn.

Free path. the Hugging Face courses, the open-source agents above with their free tiers, and every article in this guide cost nothing. Only the API usage for your builds costs money.

WEEK 1 OF 10

The Internals of Coding Agents

Core ≈ 3 h 20 min | 6 videos | 4 criteria

FOCUS what an LLM actually is and what the agent loop looks like under the hood; the core tool set (read, write, edit, bash) and how tasks flow through it; how production coding agents structure their system prompts and tool definitions.

YOU BUILD A 300–500 line terminal agent with four tools and full logging

Core material ≈ 3 h 20 min

1 Video 59 min

▸ Intro to Large Language Models

Andrej Karpathy

The best compact conceptual foundation. If you have the time, watch his longer ▸ Deep Dive into LLMs like ChatGPT (3 h 31 min) instead for a fuller foundation.

2 Article + code ≈ 60 min with the code

How to Build an Agent

Thorsten Ball

A small, legible coding agent with the essential tool loop. Use it as the model for this week's build.

3 Article 25 min

Building Effective AI Agents

Anthropic

The canonical text on workflows versus agents and the augmented-LLM loop. Barry Zhang's talk below is the video form.

4 Engineering article 20 min

Unrolling the Codex agent loop

OpenAI

A production-oriented explanation of the loop and its design tradeoffs.

5 Talk 15 min

▸ How We Build Effective Agents

Barry Zhang of Anthropic

The minimal loop, tool use, risk, verification, and debugging from inside an agent's limited context.

Production prompts and tool definitions

Read real system prompts, not summaries of them. These are public and readable:

Link files named `gpt_5_2_prompt.md`, `gpt_5_codex_prompt.md`, and similar

codex-rs/core

OpenAI Codex CLI is open source. Read the model prompts in

And the compaction prompt under `codex-rs/prompts/templates/compact/`.

Link

Gemini CLI

Google's

Keeps its system prompt in `packages/core/src/core/prompts.ts` and its MCP prompts in `packages/core/src/prompts/`.

Link

opencode

Is a third open-source coding agent whose prompts and tool schemas you can diff against the two above.

Video 1 h 06 min

▸ How Claude Code Works

An independent workshop on prompt-driven architecture, tool calls, subagents, permissions, and evaluations. The best public walkthrough of a production agent's prompt and tool design.

Optional foundation

Video 27 min

▸ Transformers, the tech behind LLMs

3Blue1Brown

The clearest visual explanation of attention.

Link

The Illustrated Transformer

Jay Alammar

The written companion.

Video 2 h 11 min

▸ How I use LLMs

Andrej Karpathy

Practical model use, tools, and limitations.

Chip Huyen, *AI Engineering* — read the sections on foundation models, evaluation, and application architecture.

Deeper material

Link

How to build a coding agent: free workshop

Geoffrey Huntley

A second from-scratch build with a different design.

Link

mini-swe-agent

A 100-line agent that scores over 74% on SWE-bench Verified. Read it after your own build to see what you over-engineered.

Link PDF, 30 min

A practical guide to building agents

OpenAI

.

Video 31 min

▸ Building Effective Agents with LangGraph

LangChain

Routing, parallelization, orchestrator-worker, and evaluator-optimizer patterns.

Additional video track

Video 13 min

▸ What's next for AI agentic workflows

Andrew Ng

Reflection, tool use, planning, and multi-agent collaboration as the four agentic patterns.

Video 17 min

▸ Software Development Agents: What Works and What Doesn't

Robert Brennan of OpenHands

The editor, terminal, browser, sandbox, and action loop.

Build

Build a terminal coding agent in roughly 300–500 lines. Give it four tools: list files, read a file, edit a file, and run a shell command. Log every model response, tool call, result, token count, and stop condition. Test it on three tiny repository tasks and manually classify every failure as model, context, tool, or control-loop failure.

Then read one production system prompt (Codex or Gemini CLI) end to end and annotate it: which lines set persona, which set safety boundaries, which shape tool selection, which handle stopping. Compare it with your own prompt.

Done when

- [ ] Your agent completes at least one of the three tasks end to end with all four tools exercised.
- [ ] Every run has a log with per-turn token counts and the final stop reason.
- [ ] A failure table exists with at least five rows, each classified as model, context, tool, or control loop.
- [ ] One annotated production prompt is checked into your notes with at least ten annotations.

WEEK 2 OF 10

Advanced Context Engineering

Core ≈ 3 h 50 min | 9 videos | 4 criteria

FOCUS advanced prompting techniques and when each applies; RePPIT (Research, Propose, Plan, Implement, Test) and spec-driven development; MCP fundamentals (servers, clients, tools, transport); designing tools for agent ergonomics.

YOU BUILD A one-page spec, one feature shipped through RePPIT, and an MCP server with 2–4 tools

Core material ≈ 3 h 50 min

1 Video/podcast 1 h 33 min

▸ Context Engineering

The Pragmatic Engineer with Dex Horthy

Context, harnesses, loops, research/plan/implement workflows, compaction, and software factories. The article and transcript are useful for notes.

2 Article 25 min

Effective context engineering for AI agents

Anthropic

The canonical framing of context as a finite resource: system prompts, tools, examples, retrieval, and compaction.

3 Video 29 min

▸ Prompting for Agents

Anthropic

How prompting changes when the model runs in a loop with tools.

4 Article 25 min

Writing effective tools for agents

Anthropic

Tool names, descriptions, boundaries, results, and evaluation.

5 Specification 30 min for the architecture, transports, and tools sections

Specification

Model Context Protocol

Read the primary source before any tutorial.

6 Documentation

Spec Kit

GitHub

And its Agentic SDD workflow (20 min) — a concrete implementation of spec-driven development.

Deeper material

Course units 0–2, roughly 6–8 h; spread it across Weeks 2 and 3

MCP Course

Hugging Face

Free, hands-on foundations, continuing into the end-to-end MCP application unit.

Link

Context Engineering for AI Agents: Lessons from Building Manus

Manus

KV-cache hit rate, file system as context, and keeping failures in the trace.

Link

How Long Contexts Fail

Drew Breunig

And Chroma, Context Rot — the evidence that more context is not free.

Link

Advanced Context Engineering

HumanLayer

And the 12-Factor Agents principles.

Link

Specs Are the New Source Code

Ravi Mehta

And the Kiro specs docs — the product side of spec-driven development.

Link

MCP Inspector

Tooling

For testing servers; the MCP Registry for publishing and discovery.

Video 10 min

▸ How I use Claude Code for real engineering

Matt Pocock

Planning, clarifying questions, phased execution, and context-window management.

Additional video track

Advanced prompting 1 h 16 min

▸ AI Prompt Engineering: A Deep Dive

Anthropic

, and the shorter ▸ Prompting 101 (24 min).

What still works 1 h 37 min

▸ AI prompt engineering in 2025: What works and what doesn't

Sander Schulhoff on Lenny's Podcast

Evidence-based prompting, few-shot, decomposition, and prompt injection.

Research, plan, implement 21 min

▸ No Vibes Allowed: Solving Hard Problems in Complex Codebases

Dex Horthy

Targeted research, deliberate compaction, concrete plans, and human review.

Long-horizon context 39 min

▸ Context Engineering Our Way to Long-Horizon Agents

Harrison Chase of LangChain

.

Spec-driven development 1 h 04 min

▸ Spec-Driven Development: Agentic Coding at FAANG Scale and Quality

Al Harris of Amazon Kiro

Requirements, acceptance criteria, design artifacts, tasks, and steering documents.

MCP workshop · 1 h 44 min

▸ Building Agents with Model Context Protocol

Mahesh Murag of Anthropic

The full workshop from the protocol's authors.

MCP video course · ≈ 1 h 30 min

MCP: Build Rich-Context AI Apps

DeepLearning.AI and Anthropic

Architecture, servers, clients, tools, resources, prompts, remote deployment, and Inspector-based testing.

Quick architecture tour · 9 min

▸ Model Context Protocol: A Deep Dive

Gaurav Sen

.

Build

Choose one real feature for your repository. Write a one-page spec containing objective, constraints, out-of-scope items, acceptance checks, edge cases, and a test plan. Follow Research → Propose → Plan → Implement → Test without skipping a stage, and keep the artifact from each stage.

Then build one MCP server with two to four narrow tools. Exercise it in MCP Inspector first, then from your agent. Test happy paths, invalid parameters, huge results, timeouts, and permission failures. Revise each description until a fresh agent consistently selects the right tool and passes valid arguments.

Done when

- [ ] The spec exists, and each RePPIT stage produced a saved artifact (research notes, proposal, plan, diff, test results).
- [ ] The MCP server passes an Inspector session covering every tool with valid and invalid input.
- [ ] A fresh agent selects the correct tool on five out of five scripted requests.
- [ ] Cost and token counts for the feature are recorded in your notes.

WEEK 3 OF 10

Agent Skills and CLI

Core ≈ 3 h 40 min | 3 videos | 4 criteria

FOCUS what skills are and how `SKILL.md` plus scripts encode a workflow; web skills and extending agent capability beyond the repository; working effectively from the CLI.

YOU BUILD A packaged skill with a helper script, plus a browser-driven web skill

Core material ≈ 3 h 40 min

1 Course unit ≈ 60 min

Agent Skills

Hugging Face

The best structured introduction to portable, progressively disclosed skills.

2 Article 20 min

Equipping agents for the real world with Agent Skills

Anthropic

, with the public skills repository as worked examples.

3 Specification + tutorial 30 min

Agent Skills Quickstart

.

4 Video 45 min

▸ The Beginner's Guide to Coding with Cursor

Lee Robinson of Cursor

Typed languages, linting, formatting, tests, branch review, and parallel background work.

5 CLI foundation ≈ 60 min with exercises

The Missing Semester 2026: Course Overview and Introduction to the Shell

MIT

Shell navigation, composition, scripting, streams, permissions, and safe Bash practices.

6 Standard 10 min

AGENTS.md

Read the examples and conventions now; repository instructions become the main topic in Week 4.

Tools and references

Link

Command Line Interface Guidelines

How to design CLIs that agents (and humans) can drive reliably. Apply it to your skill's helper script.

Link

Playwright MCP

Web skills tooling

, Chrome DevTools MCP, and browser-use.

Link

Common workflows

Claude Code

Git worktrees, parallel sessions, and resuming conversations from the CLI.

Additional video track

Hands-on skills workshop 1 h 19 min

▸ Skill Issue: How We Used AI to Make Agents Actually Good at Supabase

Pedro Rodrigues of Supabase

Builds and evaluates `SKILL.md`, supporting scripts, progressive disclosure, and routing behavior.

Web skills 18 min

▸ Bringing Agents onto the World Wide Web

Paul Klein of Browserbase

Browser-agent harnesses, network interception, WebMCP, Playwright CLI, memory, and reusable website skills.

Concept explainer 35 min

▸ How AI agents & Claude skills work (Clearly Explained)

Greg Isenberg

.

Long form 2 h 29 min

▸ Future of Programming with AI

Lex Fridman with the Cursor team

Editor design, speculative edits, and how the Cursor founders think about agents.

Build

Find a workflow you have repeated at least three times: release notes, database migration checks, API endpoint scaffolding, dependency upgrades, or UI regression checks. Package it as a skill with:

- A short `SKILL.md` that tells the agent when to use it.
- Progressive disclosure: keep rare details in linked references.
- One deterministic helper script that follows the CLI guidelines above.
- A fixture or sample input.
- A pass/fail verification command.

Run it from a fresh agent session on two different inputs. Record where the instructions were ambiguous or overloaded the context. Then add one web skill: a browser-driven check (Playwright MCP or Chrome DevTools MCP) that verifies something in your running app.

Done when

- [ ] The skill triggers correctly from a fresh session on two inputs without extra coaching.
- [ ] The helper script has a documented exit code and runs in under ten seconds.
- [ ] The web skill drives a real browser and reports pass/fail on your app.
- [ ] Your notes list every ambiguity you fixed in `SKILL.md` and why.

WEEK 4 OF 10

Customizing Your Agent and Repository

Core ≈ 3 h 40 min | 6 videos | 4 criteria

FOCUS `CLAUDE.md` and `AGENTS.md`, what to put where; hooks for lint gates, test runs, and guardrails; subagent patterns (planner / implementer / reviewer).

YOU BUILD Repository instructions, two hooks, and a planner / implementer / reviewer split

Core material ≈ 3 h 40 min

1 **Article** 30 min

Claude Code best practices

Anthropic

The canonical guide to `CLAUDE.md`, tool allowlists, workflows, and multi-Claude patterns.

2 **Article** 15 min

Steering Claude Code: skills, hooks, rules, subagents, and more

Anthropic

When to use each mechanism.

3 **Course units**

Sub-agents

Hugging Face

And Hooks (≈ 90 min).

4 **Video** 28 min

▸ Mastering Claude Code in 30 minutes

Anthropic

Boris Cherny's own walkthrough of the configuration surface.

5 **Video** 50 min

▸ Inside Claude Code With Its Creator

Boris Cherny

Terminal design, `CLAUDE.md`, teams, subagents, plan mode, and the future of coding.

Tools and references

Link

Hooks reference

Claude Code docs

And Create custom subagents — the exact event names, matchers, and frontmatter you need for the build.

Link

How to write a great agents.md: lessons from over 2,500 repositories

Repository instruction files: GitHub

; Real Python, How to Write an AGENTS.md File; Agentic AI Foundation, Writing an Effective AGENTS.md.

Link

Don't Build Multi-Agents

The subagent debate: Cognition

Versus Anthropic, How we built our multi-agent research system. Read both before designing your three roles.

Link

How Anthropic teams use Claude Code

Anthropic

Internal adoption patterns by team.

Link

Claude Code in Action

Free courses: Anthropic Academy

; DeepLearning.AI, Claude Code: A Highly Agentic Coding Assistant.

Additional video track

Full workflow 1 h 37 min

▸ AI Coding Workflow: From Product Idea to Tested Implementation

Matt Pocock

Human alignment, PRDs, vertical slices, unattended agents, QA, review, and parallelization.

Creator, short form · 18 min

▸ Claude Code & the evolution of agentic coding

Boris Cherny

.

Engineers' perspective · 1 h 10 min

▸ The Secrets of Claude Code From the Engineers Who Built It

Every

, and the Latent Space interview Claude Code: Anthropic's Agent in Your Terminal with Boris Cherny and Cat Wu.

Hooks in practice · 30 min

▸ I'm HOOKED on Claude Code Hooks: Advanced Agentic Coding

IndyDevDan

Pre/post tool hooks, logging, and stopping a destructive command.

Advanced configuration

Claude Code Advanced Patterns: Subagents, MCP, and Scaling to Real Codebases

Anthropic

Hooks, orchestration, guardrails, internal tools, and context strategies.

Practitioner perspective

5 h 15 min; watch the programming-with-agents, vibe-coding-versus-engineering, agent setup, model, and harness chapters only

▸ Future of Programming, AI, Agentic Engineering, Vibe Coding and Linux

DHH with Lex Fridman

. The timestamped transcript makes selective viewing easy.

Build

Add a concise repository instruction file. Treat it as a map, not an encyclopedia: architecture entry points, commands, conventions, safety boundaries, and links to deeper docs. Add one deterministic hook that runs a fast lint or test gate, and one that blocks a dangerous command.

Create three roles (planner, implementer, and independent reviewer) and give each a narrow contract. Run the same feature once with a single agent and once with the three-role workflow. Compare elapsed time, tokens, defects found, and amount of human intervention.

Done when

- [ ] The instruction file is under 150 lines and a fresh agent can run setup, tests, and lint from it alone.
- [ ] The lint/test hook blocks a deliberately broken commit; the safety hook blocks a deliberately dangerous command.
- [ ] The single-agent versus three-role comparison table exists with time, tokens, defects, and interventions.
- [ ] You wrote one paragraph on whether the subagent split paid for itself, citing the numbers.

WEEK 5 OF 10

Agent-Ready Codebases

Core ≈ 3 h 30 min | 3 videos | 4 criteria

FOCUS what makes a repository agent-ready: structure, docs, tests, and checks; scoring and auditing readiness; common gaps that block agents in real repositories.

YOU BUILD A scored readiness audit and the fixes that make a fresh agent productive

Core material ≈ 3 h 30 min

1 **Engineering case study** 30 min

Harness engineering: leveraging Codex in an agent-first world

OpenAI

Arguably the single most important reading for this week: repository legibility, structured documentation, machine-enforced invariants, and progressive disclosure.

2 **Practical guide** ≈ 60 min

Repository Harnesses for AI Coding Agents

Adobe

A free, systematic guide to making a repository navigable and verifiable by agents.

3 **Essay** 20 min

Harness Engineering

Martin Fowler

.

4 **Videos** 16 min

▸ Making Codebases Agent-Ready

Eno Reyes of Factory

And ▸ Building Reliable Agentic Systems (18 min) — mechanical verification, linters, end-to-end tests, interface documentation, planning, grounding, and human oversight.

5 **Evals primer** 20 min

Your AI Product Needs Evals

Hamel Husain

Start the evaluation habit here; Week 10 depends on it.

6 Measuring readiness 25 min

▸ The ROI of AI: Why You Need Eval Frameworks

Beyang Liu of Sourcegraph

Rigorous evaluations, repository context, engineering KPIs, and avoiding productivity theater.

Deeper material

Link

Unlocking the Codex harness

OpenAI

.

Video

▸ Context Engineering

Revisit Dex Horthy's

Interview, particularly the harness, loop, and software-factory chapters.

Link

Diátaxis

A documentation structure (tutorials, how-tos, reference, explanation) that agents navigate well.

Link

Context Rot

Chroma

Why a lean, well-indexed repository beats dumping everything into context.

Additional video track

Tests as the harness 1 h 15 min

▸ TDD, AI agents and coding

Kent Beck with The Pragmatic Engineer

Why test-driven development becomes a superpower when agents write the code.

Harness design 23 min

▸ When to Build Your Own Agent Harness

Harrison Chase of LangChain

.

Build

Audit your repository on five dimensions: discoverability, setup, feedback speed, architecture enforcement, and recovery from failure. Score each from 1 to 5 with written evidence. Improve it until a fresh clone can be understood, installed, tested, and changed without tribal knowledge.

At minimum, add a one-command setup, a fast deterministic test path, a short architecture map, local conventions near the code they govern, and a machine-enforced architectural boundary. Give the repository to a fresh agent with one issue and no extra coaching; document every place it gets lost. Re-score afterwards.

Done when

- [ ] Before and after readiness scores exist for all five dimensions with evidence.
- [ ] One command sets up a fresh clone; one command runs the deterministic test path in under two minutes.
- [ ] One architectural boundary is enforced by a check that fails CI when violated.
- [ ] The fresh-agent trial is logged with every point of confusion and the fix you applied.

WEEK 6 OF 10

Agentic Code Review

Core ≈ 3 h 30 min | 1 videos | 4 criteria

FOCUS what AI review catches well and what it misses; review architectures and custom rules; fitting AI review into a team's pull-request workflow.

YOU BUILD A severity-ordered review rubric wired to pull requests, measured on five PRs

Core material ≈ 3 h 30 min

1 **Canonical guide**

Engineering Practices: Code Review

Google

And the Reviewer Guide (60 min).

2 **Free book chapter** 45 min

Chapter 9: Code Review

Software Engineering at Google

.

3 **Talk** 10 min

▸ AI-powered entomology: lessons from millions of AI code reviews

Tomas Reimers of Graphite

, with Graphite's written AI code review implementation and best practices (20 min).

4 **Review architecture** 16 min

▸ How to Kill the Code Review

Ankit Jain

A five-layer trust model combining specifications, reusable guardrails, deterministic checks, executable test plans, previews, and human alignment.

5 **What automation cannot replace** 20 min

▸ Understanding Is the New Bottleneck

Geoffrey Litt

Review as architectural understanding, mentorship, and coordination rather than only correctness checking.

6 Integrations 15 min

About GitHub Copilot code review

GitHub

And Anthropic, Claude Code GitHub Actions (15 min) — the two most common ways to put a reviewer agent on a pull request.

From Cognition

Link

Don't Build Multi-Agents

Cognition

Context sharing and why reviewer agents need the full trace.

Video 32 min

▸ DeepWiki: The GitHub Encyclopedia

Latent Space with Cognition

Codebase understanding as a product; the same understanding a reviewer needs.

Link

SWE-bench technical report

Cognition

And Devin: Coding Agents 101.

Deeper material

Link

AI-Assisted Assessment of Coding Practices in Modern Code Review

Research paper

And Google Research, Resolving code review comments with ML.

Rules and feedback loops 10 min

▸ Your Coding Agent Doesn't Always Follow Your Rules

Hooks, deterministic checks, asynchronous verification, reviewer agents, and LLM-as-judge tradeoffs.

Build

Create a review rubric ordered by severity: correctness, security, data loss, concurrency, compatibility, tests, maintainability, then style. Require the reviewer agent to cite exact lines, explain the failure scenario, and propose the smallest fix. Use a model/session that did not write the change. Wire it to your pull requests with one of the integrations above.

Evaluate the reviewer on at least five pull requests, including one deliberately seeded with subtle bugs. Label every comment true positive, false positive, duplicate, or low-value. Track acceptance rate and escaped defects. A human still owns merge approval.

Done when

- [ ] The rubric is checked in and the reviewer agent runs automatically on pull requests.
- [ ] Five reviewed pull requests have every comment labeled, with precision computed.
- [ ] The seeded-bug pull request report states which planted bugs were caught and which escaped.
- [ ] One paragraph records what the reviewer systematically misses and what deterministic check now covers it.

WEEK 7 OF 10

Security

Core ≈ 3 h 50 min | 2 videos | 4 criteria

FOCUS SAST/SCA, dependency and secret-leak vulnerabilities; prompt injection and agent-specific attack surfaces; agent-assisted triage and remediation.

YOU BUILD A threat model, SAST / SCA / secret scans in CI, and a prompt-injection test

Core material ≈ 3 h 50 min

1 **Threat catalogs** 30 min

OWASP MCP Top 10

And OWASP Top 10 for LLM Applications (30 min).

2 **Agent-specific framing** 10 min

The lethal trifecta for AI agents

Simon Willison

And Prompt injection explained (talk with transcript, 15 min).

3 **Real exploit** 15 min

GitHub Copilot: Remote Code Execution via Prompt Injection (CVE-2025-53773)

Embrace The Red

A coding-agent attack chain end to end.

4 **Video** 45 min

▸ When AI Writes Code: Rethinking App Security

Isaac Evans of Semgrep

AI-generated vulnerabilities, SAST, security assistants, feedback loops, and risks created by coding agents.

5 **Hands on** 20 min

Finding vulnerabilities in modern web apps using Claude Code and OpenAI Codex

Semgrep

Agent-assisted triage in practice.

6 Coding-agent threat model 14 min

▸ Safety and Security for Code-Executing Agents

Fouad Matin of OpenAI

Remote code execution, prompt injection, exfiltration, containers, network restrictions, approvals, and OS-level sandboxing.

7 Hands-on labs

Web Security Academy

PortSwigger

Do the Web LLM attacks labs, including indirect prompt injection (≈ 60 min for two labs).

Tools and references

Link

Semgrep

SAST

And CodeQL. Secrets: gitleaks. Dependencies/SCA: OSV-Scanner. Run all three classes in the build.

Link

Making Claude Code more secure and autonomous with sandboxing

Anthropic

And the claude-code-security-review GitHub Action.

Link

MCP Tool Poisoning

OWASP

And the original Invariant Labs disclosure, MCP Security Notification: Tool Poisoning Attacks.

Link

Agentic AI Threats and Mitigations

OWASP

And the broader OWASP GenAI Security Project.

Link

Design Patterns for Securing LLM Agents against Prompt Injections

Research

; NIST, Strengthening AI Agent Hijacking Evaluations.

Additional video track

Practical sandboxing 38 min

▸ Why and How You Need to Sandbox AI-Generated Code

Cloudflare's Harshil Agrawal

Capabilities, secrets, networking, cleanup, isolation surfaces, and indirect prompt injection.

Lab companion 12 min

▸ Web Security Academy series introduction

Rana Khalil

.

Book

Link

Threat Modeling: Designing for Security

Adam Shostack

Use the first edition; the second edition, retitled for an AI world, is announced for February 2027.

Build

Threat-model the entire agent workflow: user input, repository content, retrieved web pages, tools, credentials, MCP servers, generated commands, logs, and deployment. Apply least privilege, explicit allowlists, secret isolation, sandboxing, and human approval for irreversible actions.

Run SAST (Semgrep or CodeQL), dependency/SCA (OSV-Scanner), and secret scans (gitleaks) in CI. Plant a harmless indirect-prompt-injection string in an untrusted fixture and verify that the agent treats it as data rather than instructions. Write a short incident playbook for credential exposure, malicious tool output, and runaway cost.

Done when

- [ ] A threat model document lists every trust boundary in the agent workflow with a mitigation per boundary.
- [ ] SAST, SCA, and secret scanning run in CI and block on high-severity findings.
- [ ] The planted injection string is demonstrably ignored, with the transcript saved.
- [ ] The incident playbook covers the three scenarios with an owner and first three steps for each.

WEEK 8 OF 10

Background Agents

Core ≈ 3 h 30 min | 4 videos | 4 criteria

FOCUS asynchronous, cloud-delegated agents; managing fleets of parallel agents; issue-to-PR pipelines and triggers from Slack, Linear, and GitHub.

YOU BUILD An issue-to-PR flow with isolation, budgets, checkpoints, and retries

Core material ≈ 3 h 30 min

1 The products

Codex cloud

OpenAI

; Anthropic, Claude Code on the web; Cursor, Cloud Agents; GitHub, Managing issues and pull requests with the Copilot coding agent (≈ 20 min each; compare their trigger, isolation, and approval models).

2 Orchestration case study

Open-sourcing Symphony

OpenAI

And the Symphony specification (≈ 60 min) — issue-to-PR orchestration as a specification.

3 Video 37 min

▸ Cloudflare's AI Engineering Stack: Assisted to Delegated

Rajesh Bhatia of Cloudflare

How Cloudflare moved from assisted coding to delegated agents on its own platform.

4 Durable execution 15 min

▸ Scaling AI Agents Without Breaking Reliability

Preeti Somal of Temporal

Persistent state, orchestration, visibility, automatic recovery, human interaction, and parallel work.

5 Fleet operations 9 min

▸ I Run a Fleet of AI Agents Across Three Machines: Here's What Broke

Kyle Jaejun Lee

Attention limits, durable context, blocking approvals, routing, conflicting work, and multi-machine recovery.

Tools and references

Link

Agents SDK

Cloudflare

And Sandbox SDK — a platform for stateful agents and isolated execution.

Link

Durable AI Agent tutorial

Temporal

And the OpenAI Agents integration concepts.

Link

Code Mode: give agents an entire API in 1,000 tokens

Cloudflare

A tool-efficiency technique for fleets.

Link

Jules

Google

A third cloud agent to compare.

Link

Common workflows

Claude Code

Git worktrees for parallel local agents.

Additional video track

Build your own harness 1 h 52 min

▸ Claude Agent SDK: Full Workshop

Thariq Shihipar of Anthropic

The SDK that Claude Code itself runs on, for building a custom background agent.

Where agents are going 22 min

▸ Building the future of agents with Claude

Anthropic

.

Infrastructure economics

1 h 23 min; the relevant chapters are long-running agents at `05:32`, the inference stack at `15:12`, throughput versus latency at `20:03`, and transformer hardware at `33:19`

▸ Why AI Is About to Get 1000x Cheaper

Neil Movva

. Treat “1000x” as an ambition, not a demonstrated forecast.

Build

Build an issue-to-pull-request background flow. Each job gets an isolated worktree or container, a strict budget, a checkpoint, deterministic validation, and a resumable state. Add retries with bounded backoff and prevent two agents from editing the same work item simultaneously.

Run three jobs in parallel: a small feature, a bug fix, and a documentation task. Interrupt one midway and demonstrate that it can resume or fail cleanly. Agents may open pull requests; only a human may merge.

Done when

- [ ] Three parallel jobs ran from three issues and each opened a pull request without editing the others' files.
- [ ] The interrupted job either resumed from its checkpoint or failed with a clear state, and the log proves which.
- [ ] Each job stayed under its budget, and the budget breach path was tested at least once.
- [ ] A comparison note ranks the hosted products above on trigger, isolation, and approval model.

WEEK 9 OF 10

Building an AI-Native Team

Core ≈ 3 h 40 min | 1 videos | 4 criteria

FOCUS MCP portals and centralized, permissioned tool access; LLM gateways, model routing, and cost optimization; organization-wide adoption patterns.

YOU BUILD A model gateway, an MCP portal, and a one-page adoption policy

Core material ≈ 3 h 40 min

1 Talk 18 min

▸ Agentic SDLC: Building Blocks for Uber's Software Factory

Uber

Model gateways, agent identity, PII redaction, MCP gateways, tool access, remote environments, context graphs, and pre-CI validation.

2 MCP portals 18 min

▸ Gateways Are All You Need

Anthropic's Karan Sampath

And Tobin South, ▸ What Does Enterprise-Ready MCP Mean? (14 min) — identity, access policy, observability, provisioning, oversight, and data-loss prevention.

3 Architecture 25 min

Scaling MCP adoption: reference architecture for enterprise deployments

Cloudflare

.

4 Authorization

Enterprise-Managed Authorization

MCP

And the technical specification (20 min).

5 Evidence on adoption 60 min

2025 State of AI-assisted Software Development

DORA

AI amplifies the strengths and weaknesses of the surrounding engineering system. Pair it with the counter-evidence: METR, Measuring the Impact of Early-2025 AI on Experienced Open-Source Developer Productivity (20 min).

6 Video 42 min

▸ The Future of Software Creation

Amjad Masad of Replit

Agent infrastructure, secure execution, autonomy levels, and how cheap software creation may reshape teams.

Tools and references

Link

LiteLLM AI Gateway

Gateways for the build

(open source) and Cloudflare AI Gateway. Routing primer: Vercel, Six LLM routing strategies.

Link

MCP Registry

Internal registries: the

.

Link

AI Capabilities Model

Adoption evidence: DORA

; Stack Overflow, 2025 Developer Survey: AI; OpenAI, How OpenAI uses Codex (PDF); Anthropic, How Anthropic teams use Claude Code.

Additional video track

Adoption playbook 18 min

▸ Building an Autonomous Engineering Org

A maturity model covering champions, repository readiness, delegation, review bottlenecks, isolation, and organizational impact.

Book

Link

Accelerate

Nicole Forsgren, Jez Humble, and Gene Kim

Useful for evaluating whether AI adoption improves delivery rather than merely increasing output volume.

Build

Put your model calls behind one gateway or proxy (LiteLLM or Cloudflare AI Gateway). Add per-agent identity, budgets, model routing, fallback behavior, cost and latency logs, and redaction rules. Create a small MCP portal with allowlisted tools, explicit scopes, audit logs, and revocable credentials.

Write a one-page team adoption policy covering approved data, mandatory human decisions, incident ownership, review requirements, and what metrics matter. Prefer change-failure rate, lead time, review burden, cost, and verified task success over lines of code.

Done when

- [ ] Every model call in your system goes through the gateway; a direct call is blocked or logged as a violation.
- [ ] Routing sends at least one class of request to a cheaper model, and the cost log shows the saving.
- [ ] The MCP portal exposes only allowlisted tools, and revoking a credential cuts off access within one run.
- [ ] The adoption policy fits on one page and names an owner for incidents.

WEEK 10 OF 10

The Software Factory + The Future

Core ≈ 3 h 30 min | 5 videos | 4 criteria

FOCUS self-running, self-improving software systems; running and securing agents post-deployment; where AI software engineering goes next.

YOU BUILD A traced end-to-end factory with an eval set and a controlled improvement loop

Core material ≈ 3 h 30 min

1 Keynote 39 min

▸ Software Is Changing (Again)

Andrej Karpathy

.

2 Current case study 20 min

The self-improvement loop in a software factory

Warp

.

3 Critical counterargument 19 min

▸ Harness Engineering Is Not Enough: Why Software Factories Fail

Dex Horthy

Brownfield maintainability, weak review, incidents, and architectural decay under high-volume agent output.

4 Post-deployment operations 25 min

▸ Always-on agents run production without the on-call tax

, and Google SRE, Postmortem Culture: Learning from Failure (30 min) — the operating discipline that agent-run systems still need.

5 Tracing and evals 19 min

▸ How to Debug AI Agents: Tracing, Observability & Evals

Arize AI

And Arize Phoenix, Your First Traces (30 min hands-on).

6 System design

Symphony article

Revisit OpenAI's

And specification with the factory build in mind (20 min).

Tools and references

Evals course ≈ 2 h

Evaluating AI Agents

DeepLearning.AI

Tracing, component and trajectory evaluation, LLM judges, and production monitoring.

Link

Langfuse

Observability

(open source alternative to Phoenix) and the OpenTelemetry GenAI semantic conventions.

Link

DSPy

Controlled improvement loops

Optimize prompts against your eval set instead of editing them by hand.

Link

SWE-bench

Benchmarks to borrow task design from

And Terminal-Bench.

Link

Introduction

Google SRE

.

Additional video track

Evals, in depth · 1 h 46 min

▸ Why AI evals are the hottest new skill for product builders

Hamel Husain and Shreya Shankar on Lenny's Podcast

Error analysis, LLM-as-judge validation, and the eval workflow.

Software-factory thesis

The Software Factory

Eno Reyes

A timestamped segment on agent readiness, deterministic systems, harnesses, feedback loops, and organizations as capital allocators for agent work.

The decade of agents · 2 h 26 min

▸ “We're summoning ghosts, not building animals”

Andrej Karpathy with Dwarkesh Patel

Why agents still cannot plan, remember, or compound knowledge, and what that means for the next decade.

Broad practitioner synthesis

▸ Future of Programming and Agentic Engineering

Revisit DHH's

, especially `3:59:24` on the future of programming. Use it for perspective, not as a technical authority.

Build

Connect the pieces into a minimal factory:

```
issue → aligned spec → isolated agent run → tests/checks → independent review → human merge
```

Trace every run. Create a fixed evaluation set of 10–20 representative tasks and record success, regressions, cost, latency, retries, and human interventions. Add one controlled improvement loop that may propose changes to prompts, skills, or tools, but cannot deploy those changes without evaluation and human approval. “Self-improving” must not mean “silently rewrites its own controls.”

Write one blameless postmortem for the worst failure the factory produced during the ten weeks.

Done when

- [ ] Every factory run is traceable from issue to merge decision in Phoenix or Langfuse.
- [ ] The evaluation set has baseline and final results for at least ten tasks.
- [ ] The improvement loop has proposed at least one change, and that change was evaluated before a human approved or rejected it.
- [ ] One postmortem exists with a timeline, root cause, and a change that prevents recurrence.

Suggested capstone

Build an **agentic maintenance system for a real repository**. It should accept a well-specified issue, create an isolated workspace, research and plan the change, implement it, run repository checks, perform an independent AI review, and open a pull request for human approval.

Deliverables

- A repository with one-command setup and a clear architecture map.
- A concise `AGENTS.md` or equivalent repository guide.
- One reusable agent skill and one MCP integration.
- Deterministic hooks, tests, and CI gates.
- A threat model and least-privilege permission design.
- A durable issue-to-PR background workflow with retries.
- Model gateway telemetry for cost, latency, routing, and failures.
- A fixed evaluation suite and a small results dashboard or report.
- A five-to-ten-minute demo and a retrospective describing failures and improvements.

Self-imposed completion gates

- A fresh clone can be set up and validated with one documented command.
- At least ten fixed evaluation tasks have baseline and final results.
- One interrupted background job demonstrably resumes or fails cleanly.

Generated changes cannot bypass tests, security checks, or human merge approval.

Every run is traceable to its issue, spec, prompts/context, tool calls, outputs, costs, and final review decision.

The short bookshelf

Do not try to read a dozen books cover to cover. These five cover the durable ideas behind this guide:

1. Link

   AI Engineering

   Chip Huyen

   Building and evaluating applications with foundation models.

2. Link

   Hands-On Large Language Models

   Jay Alammar and Maarten Grootendorst

   Practical LLM foundations with an accompanying code repository.

3. Link

   Software Engineering at Google

   Titus Winters, Tom Manshreck, and Hyrum Wright

   Free online; especially code review, testing, dependency management, and large-scale change.

4. Link

   Threat Modeling: Designing for Security

   Adam Shostack

   Security reasoning that remains useful when the attack surface includes agents and tools.

5. Link

   Accelerate

   Nicole Forsgren, Jez Humble, and Gene Kim

   Measuring whether a development system actually improves.

If you only have half the time

Keep the builds and reduce passive material. Use this minimum path:

1. Thorsten Ball's **How to Build an Agent**, then one production prompt file from the Codex repository.
2. Anthropic's **Effective context engineering** and **Writing effective tools**, plus the MCP specification.
3. Dex Horthy's **Context Engineering** interview.
4. Anthropic's **Claude Code best practices** and the hooks reference.
5. OpenAI's **Harness Engineering** case study.
6. Google's **Code Review** guide.
7. OWASP **MCP Top 10**, the **lethal trifecta**, and two PortSwigger labs.
8. OpenAI **Symphony** and the Uber **Agentic SDLC** talk.
9. DeepLearning.AI's **Evaluating AI Agents** course.

The practical work is the guide. Watching everything without building the cumulative system will teach vocabulary; implementing it will teach engineering judgment.